Module 18 - Web Scraping

Web scraping is how security analysts and threat intel teams **automatically collect data from the web** — vulnerability databases, IP reputation feeds, certificate logs, and more. This module teaches you to write Python scripts that fetch web pages and extract exactly the data you need.

> **Setup — install the libraries before starting:**
>
> `pip install requests beautifulsoup4`

Contents

- [ ] 1. What is Web Scraping?
- [ ] 2. The `requests` Library — Fetching Pages
- [ ] 3. HTML Structure — What We Are Parsing
- [ ] 4. `BeautifulSoup` — Navigating HTML
- [ ] 5. Extracting Data — Text, Attributes, Tables
- [ ] 6. Respecting Limits — `robots.txt` and Rate Limiting
- [ ] 7. Saving Results — CSV and JSON
- [ ] 8. Summary

---

1. What is Web Scraping?

When you visit a website, your browser:

1. Sends an **HTTP request** to the server asking for a page
2. The server sends back an **HTTP response** — a chunk of HTML text
3. Your browser reads that HTML and renders it as the page you see

Web scraping is doing the same thing **without the browser** — your Python script sends the request, receives the HTML, and extracts just the information it needs.

```
Your script                    Web server
     |  --- GET /vulnerabilities -->  |
     |  <-- 200 OK  (HTML text) ---   |
     |                                |
     |  parse HTML with BeautifulSoup |
     |  extract CVE IDs and scores    |
     |  save to findings.csv          |
```

Why this is relevant to cybersecurity

Security teams use scraping to automate tasks that would take hours manually:

- Pulling new CVE entries from NVD every morning
- Checking whether a company IP appears on threat intel blocklists
- Monitoring certificate transparency logs for phishing domains
- Collecting public OSINT data during a pentest reconnaissance phase

Two libraries do most of the work:

Library	What it does
`requests`	Sends HTTP requests and receives responses
`beautifulsoup4`	Parses the HTML and lets you search through it

In [ ]:
```
# Install check — run this first
# If either import fails, run: pip install requests beautifulsoup4
import requests
from bs4 import BeautifulSoup
import time
import csv
import json

print("Libraries loaded successfully.")
print(f"requests version  : {requests.__version__}")
```

---

2. The `requests` Library — Fetching Pages

`requests` is the standard Python library for making HTTP requests. The most common operation is `requests.get(url)` — ask a server for a page.

The response object

`requests.get()` returns a **Response object** with everything the server sent back:

Attribute	What it contains
`response.status_code`	HTTP status code (`200` = OK, `404` = not found, `403` = forbidden)
`response.text`	The raw HTML (or JSON) as a string
`response.content`	Raw bytes (for images, PDFs)
`response.headers`	HTTP headers the server sent
`response.url`	The final URL (after any redirects)

Common HTTP status codes

Code	Meaning	What to do
`200`	OK — page delivered	Proceed
`301/302`	Redirect	`requests` follows automatically
`403`	Forbidden	Server refuses — check if login or headers needed
`404`	Not Found	URL is wrong or page was removed
`429`	Too Many Requests	You are scraping too fast — add delay
`500`	Server Error	Server problem — retry later

In [ ]:
```
# Making a GET request — real example against a public security resource
# We use httpbin.org, a free HTTP testing service, to demonstrate without hammering

url = "https://httpbin.org/get"

response = requests.get(url, timeout=10)   # timeout=10: give up after 10 seconds

print(f"Status code : {response.status_code}")
print(f"Final URL   : {response.url}")
print(f"Content-Type: {response.headers.get('Content-Type', 'unknown')}")
print(f"Response length: {len(response.text)} characters")
```

In [ ]:
```
# Always check the status code before processing the response
# Never assume a 200 — servers lie, redirect, or block scrapers

url = "https://httpbin.org/status/404"   # deliberately returns 404

response = requests.get(url, timeout=10)

if response.status_code == 200:
    print("Success — processing content")
elif response.status_code == 403:
    print("Forbidden — this server blocks scrapers or requires authentication")
elif response.status_code == 404:
    print("Not Found — check the URL")
elif response.status_code == 429:
    print("Too Many Requests — slow down and add a delay")
else:
    print(f"Unexpected status: {response.status_code}")
```

In [ ]:
```
# response.raise_for_status() is a shortcut that raises an exception
# for any 4xx or 5xx status code — cleaner than a chain of if/elif

url = "https://httpbin.org/get"

try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()            # raises HTTPError if status >= 400
```

```
    print(f"OK — received {len(response.text)} chars")
except requests.exceptions.HTTPError as e:
    print(f"HTTP error: {e}")
except requests.exceptions.ConnectionError:
    print("Connection failed — check your network or the URL")
except requests.exceptions.Timeout:
    print("Request timed out — server too slow or unreachable")
```

Practice — `requests`

Exercise 1 — Write: Fetch the page at `https://httpbin.org/ip` (it returns your public IP address as JSON).

1. Make the GET request with a 10-second timeout
2. Check the status code — only proceed if it is `200`
3. Print the status code and the first 200 characters of `response.text`

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

Practice — Status Codes

Exercise 2 — Fix: The code below fetches a page but will crash on any non-200 response. Rewrite the request handling so it catches errors cleanly instead of crashing.

```
response = requests.get("https://httpbin.org/status/503")
content = response.text
print(f"Got {len(content)} characters")
```

In [ ]:
```
# Broken — this will crash silently or process bad data
response = requests.get("https://httpbin.org/status/503")
content = response.text
print(f"Got {len(content)} characters")
```

▶ 💡 **SOLUTION**

---

3. HTML Structure — What We Are Parsing

HTML is the language web pages are written in. It uses **tags** — keywords wrapped in angle brackets — to structure content. BeautifulSoup reads this structure so you can search it like a database.

Tags, attributes, and nesting

```html
<div class="vulnerability" id="cve-001">
  <h2>CVE-2024-1337</h2>
  <p class="score">9.8</p>
  <a href="/details/CVE-2024-1337">Details</a>
</div>
```

Part	Example	What it is
Tag	`<div>`, `<h2>`, `<p>`, `<a>`	The element type
Attribute	`class="vulnerability"`	Extra information on the tag
Content	`CVE-2024-1337`	The text inside the tag
Nesting	`<h2>` is inside `<div>`	Parent/child relationship

The tags you will encounter most

Tag	Purpose	Common attributes
`<div>`	Generic container	`class`, `id`
`<table>`, `<tr>`, `<td>`	Table structure	`class`
`<a>`	Link	`href` (the URL)
`<h1>` – `<h6>`	Headings	
`<p>`	Paragraph	`class`
`<span>`	Inline container	`class`
`<ul>`, `<li>`	Unordered list, list item	

In [ ]:

```python
# We use a mock HTML string throughout this section
# This represents a simplified vulnerability listing page
# In real scraping you would get this from response.text

mock_vulnerability_page = """
<html>
<body>
  <h1>Recent Vulnerabilities</h1>
  <div class="vuln-list">
    <div class="vuln-entry" id="cve-2024-1111">
      <h2 class="cve-id">CVE-2024-1111</h2>
      <span class="severity critical">CRITICAL</span>
      <span class="score">9.8</span>
      <p class="description">Remote code execution in OpenSSL — affects versions 3.
      <a href="/vuln/CVE-2024-1111" class="details-link">Full details</a>
    </div>
    <div class="vuln-entry" id="cve-2024-2222">
      <h2 class="cve-id">CVE-2024-2222</h2>
      <span class="severity high">HIGH</span>
      <span class="score">7.5</span>
      <p class="description">SQL injection in Apache HTTP Server — unauthenticated
      <a href="/vuln/CVE-2024-2222" class="details-link">Full details</a>
```

```
    </div>
    <div class="vuln-entry" id="cve-2024-3333">
      <h2 class="cve-id">CVE-2024-3333</h2>
      <span class="severity medium">MEDIUM</span>
      <span class="score">4.3</span>
      <p class="description">Privilege escalation in Linux kernel via faulty permis
      <a href="/vuln/CVE-2024-3333" class="details-link">Full details</a>
    </div>
  </div>
</body>
</html>
"""

print("Mock page ready — length:", len(mock_vulnerability_page), "characters")
```

---

4. `BeautifulSoup` — Navigating HTML

BeautifulSoup turns raw HTML text into a **searchable tree** of elements. Think of it as the difference between a pile of paper (raw HTML) and a filing cabinet (parsed soup).

Creating a soup object

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_text, "html.parser")
```

`"html.parser"` is Python's built-in HTML parser — it's what tells BeautifulSoup how to read the HTML.

The four main search methods

Method	What it finds	Returns
`soup.find("tag")`	First matching tag	Single element or `None`
`soup.find("tag", class_="name")`	First tag with that class	Single element or `None`
`soup.find_all("tag")`	All matching tags	List (possibly empty)
`soup.select("css selector")`	CSS selector query	List
`soup.select_one("css selector")`	CSS selector, first match	Single element or `None`

Getting content out of a tag

Method / Attribute	What it gives you
`tag.text`	All text inside the tag (stripped of sub-tags)
`tag.get_text(strip=True)`	Same, but strips extra whitespace

Method / Attribute	What it gives you
`tag["href"]`	Value of the `href` attribute
`tag.get("href")`	Same but returns `None` if attribute is missing (safer)
`tag["class"]`	Value of `class` attribute — returns a list

In [ ]:
```
# Parse the mock page and create the soup object
soup = BeautifulSoup(mock_vulnerability_page, "html.parser")

# find() — first match
page_title = soup.find("h1")
print(f"Page title tag : {page_title}")
print(f"Title text     : {page_title.text}")
```

In [ ]:
```
# find() with class_ argument (note the underscore — 'class' is a Python keyword)
first_cve = soup.find("h2", class_="cve-id")
print(f"First CVE ID: {first_cve.text}")
```

In [ ]:
```
# find_all() — returns a list of ALL matching elements
all_cve_ids = soup.find_all("h2", class_="cve-id")

print(f"Found {len(all_cve_ids)} CVE entries:")
for cve_tag in all_cve_ids:
    print(f"  {cve_tag.text}")
```

In [ ]:
```
# CSS selectors with select() — more precise and flexible
# ".class-name" selects by class
# "#id-name" selects by id
# "tag.class" selects a specific tag with that class

# All elements with class "score"
scores = soup.select(".score")
print("CVSS scores:")
for score_tag in scores:
    print(f"  {score_tag.text}")
```

In [ ]:
```
# select_one() — same as select() but returns only the first match
# Equivalent to find() but uses CSS selector syntax

first_link = soup.select_one("a.details-link")
print(f"First link text: {first_link.text}")
print(f"First link href: {first_link['href']}")
```

Practice — BeautifulSoup Basics

Exercise 3 — Write: Using the `mock_vulnerability_page` and `soup` object already defined above:

1. Find all `<div>` elements with class `"vuln-entry"` — how many are there?

2. For the first entry only, extract and print the CVE ID, severity, and CVSS score on separate lines.
3. Extract all `href` values from links with class `"details-link"` and print them as a list.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

Check Your Understanding: find() vs find_all()

Exercise 4 — Predict: Before running the cell below, predict the output.

- What does `soup.find("div")` return?
- What does `len(soup.find_all("div"))` return?
- What does `soup.find("span", class_="nonexistent")` return?

In [ ]:
```
# Predict before running
print(type(soup.find("div")))                          # what type?
print(len(soup.find_all("div")))                       # how many divs total?
print(soup.find("span", class_="nonexistent"))         # what if no match?
```

▶ 💡 **SOLUTION**

---

5. Extracting Data — Text, Attributes, Tables

Pattern: looping over all entries to build a list

The standard scraping workflow is:

1. Find all container elements (e.g. all `div.vuln-entry`)
2. Loop through them
3. For each, extract the fields you need
4. Store as a dict, append to a list

In [ ]:
```
# Full extraction — building a list of dicts from the mock page

vulnerabilities = []

for entry in soup.find_all("div", class_="vuln-entry"):
    cve_id      = entry.find("h2", class_="cve-id").get_text(strip=True)
    severity    = entry.find("span", class_="severity").get_text(strip=True)
    score_text  = entry.find("span", class_="score").get_text(strip=True)
    description = entry.find("p", class_="description").get_text(strip=True)
    detail_url  = entry.find("a", class_="details-link").get("href", "")

    vulnerabilities.append({
```

```
        "cve_id":      cve_id,
        "severity":    severity,
        "score":       float(score_text),   # convert to float for comparisons
        "description": description,
        "detail_url":  detail_url,
    })

print(f"Extracted {len(vulnerabilities)} records")
for vuln in vulnerabilities:
    print(f"  {vuln['cve_id']} | {vuln['severity']} | CVSS {vuln['score']}")
```

In [ ]:

```
# Filtering after extraction — only critical and high severity
high_priority = [v for v in vulnerabilities if v["score"] >= 7.0]

print(f"High priority vulnerabilities (CVSS >= 7.0): {len(high_priority)}")
for vuln in high_priority:
    print(f"  {vuln['cve_id']} — {vuln['severity']} — {vuln['description'][:60]}...
```

Extracting from HTML tables

Tables are extremely common in security data — CVE lists, IP blocklists, asset registers. The structure is always: `<table>` > `<tr>` (row) > `<td>` (cell) or `<th>` (header).

In [ ]:

```
# Mock HTML table — simulating an IP reputation feed
mock_ip_table = """
<html><body>
<table class="ip-reputation">
  <thead>
    <tr>
      <th>IP Address</th>
      <th>Threat Type</th>
      <th>Confidence</th>
      <th>Last Seen</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>45.33.32.156</td>
      <td>Scanner</td>
      <td>95%</td>
      <td>2024-11-01</td>
    </tr>
    <tr>
      <td>192.241.187.88</td>
      <td>C2 Server</td>
      <td>87%</td>
      <td>2024-10-28</td>
    </tr>
    <tr>
      <td>104.21.14.101</td>
      <td>Phishing Host</td>
      <td>91%</td>
      <td>2024-11-03</td>
    </tr>
```

```
  </tbody>
</table>
</body></html>
"""

soup_table = BeautifulSoup(mock_ip_table, "html.parser")

# Extract headers
headers = [th.get_text(strip=True) for th in soup_table.find_all("th")]
print("Headers:", headers)
```

In [ ]:

```
# Extract all data rows
threat_ips = []

rows = soup_table.select("tbody tr")   # only data rows, not header row

for row in rows:
    cells_in_row = row.find_all("td")
    if len(cells_in_row) == 4:         # guard: skip malformed rows
        threat_ips.append({
            "ip":          cells_in_row[0].get_text(strip=True),
            "threat_type": cells_in_row[1].get_text(strip=True),
            "confidence":  cells_in_row[2].get_text(strip=True),
            "last_seen":   cells_in_row[3].get_text(strip=True),
        })

print(f"Extracted {len(threat_ips)} threat IPs:")
for entry in threat_ips:
    print(f"  {entry['ip']:<20} {entry['threat_type']:<15} {entry['confidence']}")
```

Practice — Full Extraction

Exercise 5 — Write: A mock certificate transparency log feed is defined below. Extract all entries into a list of dicts with keys `domain`, `issuer`, and `logged_at`. Then filter and print only entries where the domain contains `"suspicious"`.

```
mock_ct_log = """
<html><body>
<table class="ct-log">
  <thead><tr><th>Domain</th><th>Issuer</th><th>Logged At</th></tr></thead>
  <tbody>
    <tr><td>corp.legitbank.com</td><td>DigiCert</td><td>2024-11-01
08:12</td></tr>
    <tr><td>suspicious-login.legitbank.com</td><td>Let's Encrypt</td>
<td>2024-11-01 09:44</td></tr>
    <tr><td>mail.legitbank.com</td><td>DigiCert</td><td>2024-11-01
10:01</td></tr>
    <tr><td>legitbank.com.suspicious-redirect.xyz</td><td>ZeroSSL</td>
<td>2024-11-01 11:22</td></tr>
  </tbody>
</table>
</body></html>
"""
```

In [ ]:
```
mock_ct_log = """
<html><body>
<table class="ct-log">
  <thead><tr><th>Domain</th><th>Issuer</th><th>Logged At</th></tr></thead>
  <tbody>
    <tr><td>corp.legitbank.com</td><td>DigiCert</td><td>2024-11-01 08:12</td></tr>
    <tr><td>suspicious-login.legitbank.com</td><td>Let's Encrypt</td><td>2024-11-01
    <tr><td>mail.legitbank.com</td><td>DigiCert</td><td>2024-11-01 10:01</td></tr>
    <tr><td>legitbank.com.suspicious-redirect.xyz</td><td>ZeroSSL</td><td>2024-11-0
  </tbody>
</table>
</body></html>
"""

# Your code here
```

▶ 💡 **SOLUTION**

---

6. Respecting Limits — `robots.txt` and Rate Limiting

Scraping is powerful, but it comes with professional and legal obligations. A script that ignores server limits is not a security tool — it is a denial-of-service attack.

`robots.txt` — the server's rulebook

Every well-maintained website has a file at `/robots.txt` that tells automated tools what they are allowed to do. As a professional, you **check and respect this file before scraping**.

```
# Example robots.txt
User-agent: *
Disallow: /admin/
Disallow: /private/
Allow: /public/
Crawl-delay: 5
```

- `Disallow` — do not scrape this path
- `Allow` — this path is explicitly permitted
- `Crawl-delay` — wait this many seconds between requests

Warning — `robots.txt` is not a technical lock

Python can fetch disallowed pages — the file relies on good faith. Ignoring `robots.txt` may violate the site's Terms of Service and, depending on jurisdiction and intent, may have

legal consequences. Always read and respect it.

In [ ]:
```
# Fetching and displaying robots.txt before starting a scrape
# This is professional practice — do it every time

def check_robots_txt(base_url):
    """Fetch and display the robots.txt for a given domain."""
    robots_url = base_url.rstrip("/") + "/robots.txt"
    try:
        response = requests.get(robots_url, timeout=10)
        if response.status_code == 200:
            print(f"robots.txt for {base_url}:")
            print("-" * 40)
            print(response.text[:800])   # show first 800 chars
        else:
            print(f"No robots.txt found (status {response.status_code}) — proceed w
    except requests.exceptions.RequestException as e:
        print(f"Could not reach {robots_url}: {e}")

check_robots_txt("https://httpbin.org")
```

Rate Limiting — `time.sleep()`

Sending requests too quickly:

- Overloads the server (effectively a DoS)
- Gets your IP blocked
- May trigger anti-bot measures

Rule of thumb: At least 1–2 seconds between requests to the same server. Check `Crawl-delay` in `robots.txt` and honour it.

In [ ]:
```
import time

# Simulated multi-page scrape with polite delays
page_urls = [
    "https://httpbin.org/get?page=1",
    "https://httpbin.org/get?page=2",
    "https://httpbin.org/get?page=3",
]

DELAY_SECONDS = 2   # be polite — 2 seconds between requests

results = []
for i, url in enumerate(page_urls, start=1):
    print(f"Fetching page {i}/{len(page_urls)}: {url}")
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        results.append({
            "url":    url,
            "status": response.status_code,
```

```
            "length": len(response.text),
        })
        print(f"  OK — {len(response.text)} chars")
    except requests.exceptions.RequestException as e:
        print(f"  Failed: {e}")

    if i < len(page_urls):                  # no delay after the last page
        print(f"  Waiting {DELAY_SECONDS}s...")
        time.sleep(DELAY_SECONDS)

print(f"\nCompleted. Fetched {len(results)}/{len(page_urls)} pages successfully.")
```

Practice — Respecting Limits

Exercise 6 — Write: Fetch `robots.txt` from `https://example.com` and print:

1. The full content of the file (if it exists and returns 200)
2. Whether the root path `/` is explicitly allowed or disallowed (check if `/` appears in the text)

Add a 1-second delay before the second request to `https://example.com/` itself, then print the status code of that second request.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

7. Saving Results — CSV and JSON

Scraped data is only useful if you save it. Two formats cover almost every use case:

Format	Best for	Python module
CSV	Tabular data (rows and columns), opening in Excel	`csv`
JSON	Nested or hierarchical data, API output	`json`

Saving to CSV

In [ ]:
```
import csv

# vulnerabilities was built earlier from the mock page
# Save it as a CSV file

output_file = "vulnerability_findings.csv"
fieldnames = ["cve_id", "severity", "score", "description", "detail_url"]

with open(output_file, "w", newline="", encoding="utf-8") as f:
```

```
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(vulnerabilities)

print(f"Saved {len(vulnerabilities)} records to {output_file}")

# Verify by reading it back
with open(output_file, "r", encoding="utf-8") as f:
    print("\nFirst 300 chars of the CSV:")
    print(f.read(300))
```

Saving to JSON

In [ ]:

```
import json

# Save the same vulnerability list as JSON
output_file = "vulnerability_findings.json"

with open(output_file, "w", encoding="utf-8") as f:
    json.dump(vulnerabilities, f, indent=2, ensure_ascii=False)

print(f"Saved {len(vulnerabilities)} records to {output_file}")

# Read back to verify
with open(output_file, "r", encoding="utf-8") as f:
    loaded = json.load(f)

print(f"\nLoaded back {len(loaded)} records")
print(f"First record: {loaded[0]}")
```

Which format to choose?

Use CSV when:

- The data is a flat table (every record has the same fields)
- The consumer is Excel, a database import, or another analyst

Use JSON when:

- The data is nested (e.g. a vulnerability with a list of affected products)
- You are building a pipeline that feeds another Python script or API
- You need to preserve types (numbers stay numbers, booleans stay booleans)

Practice — Saving Results

Exercise 7 — Write: Use the `threat_ips` list extracted from the table earlier.

1. Save it as `threat_intel.csv` with headers `ip`, `threat_type`, `confidence`, `last_seen`
2. Save the same data as `threat_intel.json`

3. Read the JSON back and print how many records it contains.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

8. Summary

Core workflow

```
1. Check robots.txt          — always, before anything else
2. requests.get(url)         — fetch the page
3. response.raise_for_status() — fail fast on errors
4. BeautifulSoup(response.text, "html.parser") — parse the HTML
5. soup.find() / find_all() / select() — locate elements
6. tag.get_text(strip=True) / tag.get("href") — extract data
7. Build list of dicts       — one dict per record
8. Save to CSV or JSON       — persist the results
9. time.sleep(N)             — be polite between requests
```

Key rules at a glance

Rule	Why
Always set `timeout=` on requests	Scripts must not hang forever
Always call `raise_for_status()`	Catch server errors before processing garbage
Always check `robots.txt`	Professional and legal obligation
Always add `time.sleep()` in loops	Avoid overloading servers and getting blocked
Use `tag.get("attr")` not `tag["attr"]`	Returns `None` instead of crashing on missing attributes
Guard `find()` results with `if result is not None`	`find()` returns `None` when nothing matches

Common mistakes

Mistake	Symptom	Fix
`tag["href"]` on a tag without `href`	`KeyError` crash	Use `tag.get("href", "")`
No `timeout` on requests	Script hangs forever	Always pass `timeout=10`

Mistake	Symptom	Fix
No delay in loop	IP blocked, 429 errors	Add `time.sleep(1)` or more
`class_` without underscore	No error — just finds nothing	`find("div", class_="name")` not `class=`
Processing a `None` find result	`AttributeError`	`result = soup.find(...); if result: ...`

Next Steps

- **01_Advanced_Scraping.ipynb** — headers, sessions, pagination, error handling, Selenium intro